

Student Kickoff Guide: Event Detection (Driblab)

A simple, ordered path to get started.

What you're building

A system that **detects and classifies football events** (passes, shots, tackles, interceptions...) from **tracking data** (the x,y positions of players and the ball, 10×/second). You also get **labelled events** for a few matches (your ground truth) and **video** (only needed if you explore the pose question).

The one rule that saves you weeks

Understand the data, align it, then detect. In that order. Most "model bugs" in this project are actually data-alignment bugs. Do not train anything in week 1.

STEP 1: Analyze the data first (the most important step)

Before any modelling, explore each file and *answer these questions yourself*. They are designed to surface every trap in this dataset.

1a. Event vocabulary: `dim_event_type.csv`

- How many event types are **defined** vs how many actually **occur** in the labelled matches?
- What is the **class distribution**? (One class dominates massively. Which, and by how much?)
- Which entries are real events vs which only ever appear as **qualifiers** on another event?
- Decide: which handful of classes are realistically worth detecting?

1b. Events: `*_events.json`

- What are the fields? Which one is the event **location**, which is the **timing**?
- What **coordinate system** are events in, and what's the numeric range?
- Are coordinates **normalized to attacking direction**? Test it: do *both* teams' shots cluster at high x, in *both* halves?
- How is **time** encoded (period, minute, second)? Does the minute reset at half-time or keep counting?
- What does `outcome` mean, and does it mean the same thing for every event type?

1c. Tracking: `*_tracking_data.jsonl`

- How is the file structured (the header line vs the per-frame lines)? How do IDs map to player names and teams?
- What **coordinate system / units / origin** do positions use? Verify against the pitch diagram (105×68, origin bottom-left).
- What's the **sampling rate**? How many players appear per frame? How are substitutes handled?
- The **vis flag**: what fraction of positions are `true`? What does `false` actually mean? (Hint: read Driblab's methodology.)
- The **ball**: what fraction of frames have it? Is there a real height (z)? Is it in the same coordinate system as the players?
- The **cam field**: what is it, and how does it relate to whether the data in that frame is reliable?

1d. Cross-file consistency (where events meet tracking)

- Do **team and player IDs** match between events and tracking? (They should. Verify it.)
- Do the **time clocks** line up, or is there drift between the event clock and the tracking clock?
- Do the **coordinate systems** match, or must you transform one to the other (and flip per half)?
- **Which matches have which assets** (video / tracking / events)? How many have all three?

Key insights you should walk away with

If your analysis is correct, you should have discovered all of these. They shape everything downstream:

1. **Two coordinate systems.** Events are normalized 0 to 100 (always attacking toward 100); tracking is physical metres (105×68) and teams switch ends at half-time. You must align them, including a per-half flip.
2. **Two time references with drift.** Events and tracking must be time-synced before labels mean anything.
3. **The ball is the key signal but is incomplete:** present roughly two-thirds of all frames (~90% during live play), missing in brief blips, and **never guessed/imputed** when unseen.
4. **~Half of player positions are AI-imputed** (`vis=false`), not observed, so less reliable, especially off the ball.
5. **Severe class imbalance:** passes dominate (~70%), the valuable events (shots, goals, fouls) are rare.
6. **Players are dots, not bodies.** No bounding box, no pose in the tracking, so the pose question requires going back to the video.

7. **Few fully-labelled matches:** favour data-efficient methods; don't expect to train large networks.
-

Suggested Steps

Once you understand the data (Step 1), follow this ordered build path. Each step is a complete, gradeable result on its own; get one working before reaching for the next.

STEP 2: Build the foundation (align + clean)

1. Convert event coordinates to the tracking pitch (scale + per-half attacking-direction flip).
2. Sync event timestamps to tracking frames; correct the drift.
3. Filter to reliable live play (`cam` present); interpolate short ball gaps.
4. Build features: ball speed/acceleration, ball-to-player distances, who has possession, player speeds.

STEP 3: Possession backbone

Assign the ball to the **nearest player** (smooth it over time). Track when possession **changes**; this is the skeleton of most events.

STEP 4: Rule-based detection (your floor, aim for this first)

Turn possession + ball movement into events with simple rules:

- pass = ball goes to a teammate; interception/tackle = an opponent gets it; shot = ball heads to goal fast; out/throw/corner = ball leaves the pitch.

Evaluate with **per-class precision/recall/F1** on a match you didn't tune on. A working rule-based detector is a complete, gradeable result.

STEP 5: Machine learning for the hard cases

For ambiguous events, take a short time-window of features and train a classifier (start simple, e.g. gradient-boosted trees). Use class weighting; **never optimize plain accuracy** (predicting "pass" every time already scores ~70%). **Target: working ML classifier by the end of week 4.**

STEP 6: (Optional) Does pose help?

Only for visually-confusable events (tackle vs block, header vs chest, fouls), only if video/bounding boxes are available. Run players near the ball through a pose model, add those features, and **measure the F1 improvement**. The measurement *is* the answer to "does pose help?".

Team organization (6-member groups)

Split into two parallel tracks: **4 on the core** (Steps 2 to 5, the tracking pipeline) and **2 on CV/pose** (Step 6). The pose pipeline is separable, so it can be built at the same time, but follow these rules so the two tracks actually meet:

1. **Everyone does Step 1 together first.** Don't split until the data is understood (~end of week 1). Shared understanding of the coordinate/time/ball realities is what keeps both tracks compatible.
2. **The CV pair's first job is de-risking, not building.** Confirm the blockers are resolved: are the **bounding boxes** shared by Driblab? Is there **GPU** access? Get a pose model running on a few sample frames. If boxes aren't available, descope pose to a small proof-of-concept on a handful of clips (full re-detection is too much for 2 people).
3. **Agree the integration contract early (by week 2).** The CV pair delivers, for each (event timestamp, player near the ball), a **pose feature vector**; the core team plugs it into the Step-5 classifier as extra features. Write this down before either side builds far.
4. **Hard integration checkpoint at ~week 5.** The core ML is done at the end of week 4, so week 5 is when pose features must start flowing into the classifier. If they aren't, the CV pair rejoins the core. The biggest risk is two tracks that never connect.
5. **The CV deliverable is the measurement.** Pose may add little (most events are passes), and that's fine. "Per-class F1 with vs without pose" is a complete, gradeable result even if the answer is "pose doesn't help much."

Core team of 4 can sub-split: data-eng/alignment · possession+rules · ML modelling · evaluation.

Common traps (don't do these)

- **✗** Modelling before aligning coordinates and time.
- **✗** Optimizing accuracy instead of per-class F1 (imbalance makes accuracy meaningless).
- **✗** Trying to detect all event types; focus on the ones that actually occur.
- **✗** Starting with pose. It's the last step, not the first.
- **✗** Trusting the ball blindly; it can be missing exactly during shots and tackles.

Rough pacing (8 weeks)

- **Wk 1:** Step 1 (data analysis). Data understood. CV pair de-risks pose (boxes/GPU/model).
- **Wk 2:** Step 2. Alignment + features clean and working.

- **Wk 3:** Steps 3 to 4. Possession + rule-based detector + first per-class evaluation. CV builds pose extraction.
- **Wk 4:** Step 5. **ML classifier for ambiguous events working by end of week 4.**
- **Wk 5 to 6:** Step 6. **Pose integration checkpoint (wk5)** + ablation; refine the core model.
- **Wk 7 to 8:** Final ablation / refinement; write-up + presentation.